Checkpoint-Trajectory Contamination Audit on Pythia

Vlad Dancau
ETH Zurich

September 7, 2026

Abstract

We test whether a benchmark item was plausibly seen during a model's pretraining by tracking a memorization signal — loss on the item minus a held-out validation baseline — across its *full public checkpoint trajectory*, rather than only its final weights, and wrap the audit in a lightweight cryptographic trail (commitment, Merkle root, signature) for non-repudiation. Run against **pythia-70m** with ARC-Easy as the candidate set and GPQA as a post-training-cutoff negative control: the raw decision rule finds nothing, masked by generic loss volatility early in training; excluding that noisy window, the same detector consistently flags roughly half of the 50 ARC-Easy items, and the flagged signal is shown to be ARC-Easy-specific rather than a shared model-wide artifact. This is real, converging evidence of contamination from one detector on one small model, not yet a calibrated detection rate.

1 Introduction

A model provider can claim clean evaluation numbers while quietly training on benchmark data, and comparing inputs against only a model's final checkpoint is a weak way to catch this: training noise accumulates over SGD steps [Jia et al., 2021], smearing out any one data point's influence by the end of training — consistent with final-checkpoint-only membership detectors performing close to random at pretraining scale [Duan et al., 2024]. Model families that publish their full checkpoint trajectory, such as Pythia [Biderman et al., 2023], avoid this problem: the signal can be read right at the moment of exposure.

2 Method

For item d at checkpoint W, define the memorization signal $M(d, W) = \mathbb{E}_{d' \sim \text{validation}}[L(d', W)] - L(d, W)$ [Choi et al., 2023]: how much better the model does on d than on held-out data, which cancels the shared "getting better at everything" trend. An item's contamination score is the largest step-to-step jump in M across the trajectory; the checkpoint where that jump occurs is the candidate exposure step. Each item's score is converted to an empirical p-value against a null calibrated from items assumed clean, and flagged under Benjamini–Hochberg false discovery rate control at $q = 0.05$ rather than an arbitrary cutoff. The audit scope and the checkpoint set are committed to (hash commitment, Merkle root) before the sweep runs and the final report is Ed25519-signed, so the process is pre-registered and non-repudiable — a lightweight academic version of this idea, not a hardened one.

3 Results

The pipeline was run against **pythia-70m**'s full 154-checkpoint trajectory, 50 ARC-Easy items as candidates and 50 GPQA items as the negative control. The unrestricted check flags nothing — GPQA's own early-training noise is large enough to mask any real signal. Restricting to checkpoints past that noisy window changes this, and the change is stable across a wide range of cutoffs (Table 1), consistently flagging 27–29 of the 50 ARC-Easy items.

Table 1: ARC-Easy items flagged (of 50) as a function of the late-checkpoint cutoff `min_step`.

`min_step`	Flagged / 50
0 (unrestricted)	0
3,000 – 30,000	≈27–29
60,000	collapses

54% of flagged items independently peak at the same checkpoint (step 43,000) — consistent with ARC-Easy entering training as one contiguous chunk. To rule out a checkpoint-wide artifact rather than real content-specific memorization, ARC-Easy's and GPQA's traces were averaged separately: ARC-Easy's average dips sharply and recovers exactly at step 43,000; GPQA's stays flat through the same window.

4 Limitations

The cutoff `min_step` was chosen after seeing that it helped, only partly mitigated by checking the result across nearby cutoffs. This is evidence from one detector on one small (70M) model, not a measured precision/recall number. The method only applies to model families that publish full checkpoint trajectories, is strongest on near-exact textual matches, and a missing spike is not proof of no contamination. The Merkle/signature layer guarantees procedural integrity of the audit, not that the checkpoints themselves are authentic.

5 Future Work

A controlled calibration experiment — continue-pretraining on known-contaminated vs. known-clean items at varying doses — would turn this case-study evidence into a measured detection rate, and repeating the real run on larger Pythia sizes would show whether the observed flag rate and checkpoint clustering are specific to this 70M model.

LLM Use Disclosure

Claude Code (Anthropic) was used to accelerate development of the accompanying notebook's implementation — writing and iterating on the pipeline code, debugging the real Colab run, and drafting this report from its results. The experimental design, the real run itself, and the interpretation of its results remain the author's responsibility.

References

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. *arXiv preprint arXiv:2304.01373*, 2023.

Dami Choi, Yonadav Shavit, and David Duvenaud. Tools for verifying neural models' training data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Michael Duan, Anshuman Suri, Niloofar Mireshghallah, Sewon Min, Weijia Shi, Luke Zettlemoyer, Yulia Tsvetkov, Yejin Choi, David Evans, and Hannaneh Hajishirzi. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841*, 2024.

Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1039–1056, 2021.